Taskademia Front-End API Guide

Plain-English reference based on the current TypeScript controllers and route files.

Base API assumption	**This guide assumes the routers are mounted as /api/auth, /api/users, /api/projects, /api/project-members, /api/search, and /api/tasks.**
Auth pattern	Most endpoints require Authorization: Bearer <accessToken>. Refresh uses an HttpOnly cookie named refreshToken sent to /api/auth/refresh.

How to read this document:

- Request body = JSON the front end sends.
- Success response = the shape the front end can expect back on success.
- Common errors = frequent response messages worth handling in UI.
- This guide describes current behavior, including a few implementation quirks.

Authentication

POST /api/auth/register

Creates a user, hashes the password, stores an email verification token, and sends a verification email.

Auth	**Public**
What it does	Creates a user, hashes the password, stores an email verification token, and sends a verification email.

Request body	{
	"email":"user@example.com",
	"password":"Str0ng!Pass",
	"displayName":"Jane Doe"
	}

Success response	201 ->{
	"message",
	"user":{
	"id",
	"email",
	"displayName",
	"isEmailVerified"
	}
	}

Common errors	400 missing fields / weak password / user exists; 500 email failed or server error
Notes	Password rules: at least 8 chars, plus 1 lowercase, 1 uppercase, 1 number, and 1 symbol.

POST /api/auth/login

Checks email + password, requires verified email, sets refreshToken cookie, and returns basic user info.

Auth	**Public**

What it does	Checks email + password, requires verified email, sets refreshToken cookie, and returns basic user info and accessToken.

Request body	{
	"email":"user@example.com",
	"password":"Str0ng!Pass"
	}

Success response	201 -> {
	"message",
	"user":{
	"id",
	"email",
	"displayName",
	"isEmailVerified"
	}
	}

Common errors	400 invalid credentials; 403 email not verified; 500 server error
Notes	

GET /api/auth/verify-email/:token

Consumes the email verification token and redirects the browser to the login page on the front end.

Auth	Public
What it does	Consumes the email verification token and redirects the browser to the login page on the front end.
Success response	Redirects to FRONTEND_URL/login?verified=true on success
Common errors	Redirects to FRONTEND_URL/login?error=invalid-token or ?error=server-error

POST /api/auth/resend-verification

Resends a verification email if the account exists and is still unverified.

Auth	Public
What it does	Resends a verification email if the account exists and is still unverified.
Request body	{"email":"user@example.com"}
Success response	200 -> {"message":"If that email exists and still needs verification, a verification link has been sent."}
Common errors	400 missing email; 500 mail send failure

POST /api/auth/refresh

Validates the refresh token cookie, rotates it, and returns a fresh access token.

Auth	Uses refreshToken cookie only
What it does	Validates the refresh token cookie, rotates it, and returns a fresh access token.
Success response	200 -> {"accessToken":"<jwt>"}
Common errors	401 missing / invalid / expired / revoked refresh token; 500 server error
Notes	Cookie is HttpOnly. In production it is secure and sameSite=none, scoped to /api/auth/refresh.

POST /api/auth/logout

Clears the stored refresh token on the user and clears the refreshToken cookie.

Auth	Usually called with refreshToken cookie present
What it does	Clears the stored refresh token on the user and clears the refreshToken cookie.
Success response	200 -> {"message":"Logged out successfully"}
Common errors	500 server error

POST /api/auth/forgot-password

Creates a password reset token and emails the reset link.

Auth	Public
What it does	Creates a password reset token and emails the reset link.
Request body	{"email":"user@example.com"}
Success response	200 -> {"message":"Password reset link sent to email."} or generic success when account does not exist
Common errors	400 missing email; 500 server error

POST /api/auth/reset-password/:token

Validates the reset token, saves the new password hash, clears reset fields, and increments tokenVersion.

Auth	Public
What it does	Validates the reset token, saves the new password hash, clears reset fields, and increments tokenVersion.
Request body	{"password":"NewStr0ng!Pass"}
Success response	200 -> {"message":"Password updated successfully. Try logging in now."}
Common errors	400 password missing or token invalid/expired; 500 server error

GET /api/auth/me

Returns the currently authenticated user.

Auth	Bearer access token required
What it does	Returns the currently authenticated user.
Success response	200 -> {"user":{"id","email","displayName"}}
Common errors	401 no token / invalid token / revoked token; 403 email not verified

Profile

GET /api/users/profile

Returns the authenticated user's profile object.

Auth	Bearer access token required
What it does	Returns the authenticated user's profile object.
Success response	200 -> `{ "displayName": "string", "aboutMe": "string", "preferredRoles": [ "string" ], "school": "string", "profilePictureUrl": "string" }`
Common errors	404 profile not found; 500 server error
Notes	This endpoint returns the raw profile object, not wrapped in { profile: ... }.

PUT /api/users/profile

Replaces the stored profile with the profile object sent by the front end.

Auth	Bearer access token required
What it does	Replaces the stored profile with the profile object sent by the front end.
Request body	`{ "profile": { "displayName": "First Last", "aboutMe": "My Bio", "preferredRoles": [ "Frontend" ], "school": "UCF", "profilePictureUrl": "" } }`
Success response	200 -> `{ "success": true, "message": "string", "profile": { "displayName": "string", "aboutMe": "string", "preferredRoles": [ "string" ], "school": "string", "profilePictureUrl": "string" } }`

Common errors	400 validation failed; 404 user not found; 500 server error
Notes	This is a full profile object under profile, not partial top-level fields.

Projects

POST /api/projects/create

Creates a project, automatically adds the creator as Owner, and optionally creates tasks from goals.

Auth	Bearer access token required
What it does	Creates a project, automatically adds the creator as Owner, and optionally creates tasks from goals.
Request body	`{ "name": "string", "description": "string", "visibility": "private", "dueDate": "2026-03-30T01:15:58.883Z", "goals": [ { "title": "string", "description": "string" } ] }`
Success response	201 -> `{ "message": "string", "project": { "_id": "string", "name": "string", "description": "string", "visibility": "string", "dueDate": "2026-03-30T01:19:59.734Z", "createdBy": {}, "createdAt": "2026-03-30T01:19:59.734Z", "updatedAt": "2026-03-30T01:19:59.734Z" } }`
Common errors	400 missing name; 401/403 auth issues; 500 server error
Notes	Each goal becomes an initial task in the project. Visibility defaults unless explicitly set.

GET /api/projects

Returns all projects where the user is an active member, with member and task counts added.

Auth	Bearer access token required
What it does	Returns all projects where the user is an active member, with member and task counts added.
Success response	200 ->

	[
	  {
	  "_id": "string",
	  "name": "string",
	  "description": "string",
	  "visibility": "string",
	  "dueDate": "2026-03-30T01:14:21.588Z",
	  "createdBy": {},
	  "createdAt": "2026-03-30T01:14:21.588Z",
	  "updatedAt": "2026-03-30T01:14:21.588Z",
	  "memberCount": 0,
	  "taskCounts": {
	  "total": 0,
	  "todo": 0,
	  "in_progress": 0,
	  "blocked": 0,
	  "done": 0
	  }
	  }
	]

Common errors	500 server error

GET /api/projects/:projectId

Returns one project if the requester belongs to it.

Auth	Bearer access token required and must be an active member
What it does	Returns one project if the requester belongs to it.

Success response	200 ->
	{
	  "_id": "string",
	  "name": "string",
	  "description": "string",
	  "visibility": "string",
	  "dueDate": "2026-03-30T01:16:18.430Z",
	  "createdBy": {},
	  "createdAt": "2026-03-30T01:16:18.430Z",
	  "updatedAt": "2026-03-30T01:16:18.430Z"
	}

Common errors	403 access denied; 404 project not found; 500 server error

GET /api/projects/:projectId/details

Returns the project, active members, tasks, and task stats in one response.

Auth	Bearer access token required and must be an active member
What it does	Returns the project, active members, tasks, and task stats in one response.

Success response	200 -> {
	  "project": {
	  "_id": "string",
	  "name": "string",
	  "description": "string",

```
    "visibility": "string",
    "dueDate": "2026-03-30T01:16:56.029Z",
    "createdBy": {},
    "createdAt": "2026-03-30T01:16:56.029Z",
    "updatedAt": "2026-03-30T01:16:56.029Z"
  },
  "members": [
    {
      "_id": "string",
      "projectId": "string",
      "userId": {},
      "role": "string",
      "permissions": {
        "canEditProject": true,
        "canManageMembers": true,
        "canCreateTasks": true,
        "canAssignTasks": true,
        "canCompleteAnyTask": true,
        "canModerateChat": true
      },
      "membershipStatus": "active",
      "joinedBy": {},
      "createdAt": "2026-03-30T01:16:56.029Z",
      "updatedAt": "2026-03-30T01:16:56.029Z"
    }
  ],
  "tasks": [
    {
      "_id": "string",
      "projectId": "string",
      "title": "string",
      "description": "string",
      "dueDate": "2026-03-30T01:16:56.029Z",
      "assignedToUserIds": [
        {}
      ],
      "createdBy": {},
      "status": "todo",
      "priority": "low",
      "tags": [
        "string"
      ],
      "roleRequired": "string",
      "completedAt": "2026-03-30T01:16:56.029Z",
      "completedBy": {},
      "createdAt": "2026-03-30T01:16:56.029Z",
      "updatedAt": "2026-03-30T01:16:56.029Z"
    }
  ],
  "stats": {
    "totalTasks": 0,
    "todo": 0,
    "in_progress": 0,
    "blocked": 0,
    "done": 0
  }
}
```


Common errors	403 access denied; 404 project not found; 500 server error

Notes	Best endpoint for a project detail page because it bundles most related data.

PUT /api/projects/:projectId

Updates selected project fields.

Auth	Bearer access token required plus edit permission
What it does	Updates selected project fields.
Request body	`{ "name": "string", "description": "string", "visibility": "private", "dueDate": "2026-03-30T01:21:13.534Z" }`
Success response	200 -> `{ "name": "string", "description": "string", "visibility": "private", "dueDate": "2026-03-30T01:17:46.167Z" }`
Common errors	403 no access or no edit permission; 404 not found; 500 server error
Notes	Current controller updates description, visibility, and dueDate. Name is typed in the body interface but is not actually updated here.

DELETE /api/projects/:projectId

Deletes the project and also deletes its tasks and project-member records.

Auth	Bearer access token required; Owner only
What it does	Deletes the project and also deletes its tasks and project-member records.
Success response	200 -> `{ "message": "string" }`
Common errors	403 access denied / only owner can delete; 404 project not found; 500 server error

Project Members

GET /api/project-members/project/:projectId

Returns all active members for a project.

Auth	Bearer access token required and must belong to project
What it does	Returns all active members for a project.

Success response	200 ->
	[
	  {
	  "_id": "string",
	  "projectId": "string",
	  "userId": {},
	  "role": "string",
	  "permissions": {
	  "canEditProject": true,
	  "canManageMembers": true,
	  "canCreateTasks": true,
	  "canAssignTasks": true,
	  "canCompleteAnyTask": true,
	  "canModerateChat": true
	  },
	  "membershipStatus": "active",
	  "joinedBy": {},
	  "createdAt": "2026-03-30T01:22:10.504Z",
	  "updatedAt": "2026-03-30T01:22:10.504Z"
	  }
	]

Common errors	403 access denied; 500 server error

POST /api/project-members/project/:projectId

Adds a member to a project.

Auth	Bearer access token required plus canManageMembers
What it does	Adds a member to a project.

Request body	{
	  "userId": "string",
	  "role": "string",
	  "permissions": {
	  "canEditProject": true,
	  "canManageMembers": true,
	  "canCreateTasks": true,
	  "canAssignTasks": true,
	  "canCompleteAnyTask": true,
	  "canModerateChat": true
	  }
	}

Success response	201 ->
	{
	  "message": "string",
	  "member": {
	  "_id": "string",
	  "projectId": "string",
	  "userId": {},
	  "role": "string",
	  "permissions": {
	  "canEditProject": true,
	  "canManageMembers": true,
	  "canCreateTasks": true,
	  "canAssignTasks": true,

	"canCompleteAnyTask": true, "canModerateChat": true }, "membershipStatus": "active", "joinedBy": {}, "createdAt": "2026-03-30T01:25:29.010Z", "updatedAt": "2026-03-30T01:25:29.010Z" } }
Common errors	400 missing userId / already associated; 403 no permission; 404 project or user not found; 500 server error
Notes	If permissions are omitted, booleans default mostly to false, except canCreateTasks defaults to true.

PUT /api/project-members/:membershipId

Updates role, permissions, or membershipStatus for an existing membership.

Auth	Bearer access token required plus canManageMembers
What it does	Updates role, permissions, or membershipStatus for an existing membership.

Request body	{
	"role": "string",
	"permissions": {
	"canEditProject": true,
	"canManageMembers": true,
	"canCreateTasks": true,
	"canAssignTasks": true,
	"canCompleteAnyTask": true,
	"canModerateChat": true
	},
	"membershipStatus": "active"
	}

Success response	200 ->
	{
	"message": "string",
	"member": {
	"_id": "string",
	"projectId": "string",
	"userId": {},
	"role": "string",
	"permissions": {
	"canEditProject": true,
	"canManageMembers": true,
	"canCreateTasks": true,
	"canAssignTasks": true,
	"canCompleteAnyTask": true,
	"canModerateChat": true
	},
	"membershipStatus": "active",
	"joinedBy": {},
	"createdAt": "2026-03-30T01:26:16.580Z",
	"updatedAt": "2026-03-30T01:26:16.580Z"
	}
	}

Common errors	400 cannot demote Owner through this endpoint; 403 access denied / no permission; 404 membership not found; 500 server error
Notes	Permission updates are partial: omitted permission fields keep their old values.

DELETE /api/project-members/:membershipId

Removes a project member. Managers can remove others; any user can remove themselves.

Auth	Bearer access token required
What it does	Removes a project member. Managers can remove others; any user can remove themselves.
Success response	200 -> {"message":"Project member removed successfully."}
Common errors	400 owner cannot be removed here; 403 no permission; 404 membership not found; 500 server error

Tasks

POST /api/tasks

Creates a task in a project.

Auth	Bearer access token required plus task-create permission
What it does	Creates a task in a project.
Request body	`{ "projectId": "string", "title": "string", "description": "string", "dueDate": "2026-03-30T01:27:39.520Z", "assignedToUserIds": [ "string" ], "status": "todo", "priority": "low", "tags": [ "string" ], "roleRequired": "string" }`
Success response	201 -> `{ "message": "string", "task": { "_id": "string", "projectId": "string", "title": "string", "description": "string", "dueDate": "2026-03-30T01:27:39.521Z", "assignedToUserIds": [ {} ],`

	"createdBy": {}, "status": "todo", "priority": "low", "tags": ["string"], "roleRequired": "string", "completedAt": "2026-03-30T01:27:39.521Z", "completedBy": {}, "createdAt": "2026-03-30T01:27:39.521Z", "updatedAt": "2026-03-30T01:27:39.521Z" } }
Common errors	400 missing projectId/title, bad assignedToUserIds, non-member assignee; 403 no permission; 404 project not found; 500 server error
Notes	assignedToUserIds must all be active project members.

GET /api/tasks/project/:projectId

Returns all tasks in a project.

Auth	Bearer access token required and must belong to project
What it does	Returns all tasks in a project.

Success response	200 ->
	[
	{
	"_id": "string",
	"projectId": "string",
	"title": "string",
	"description": "string",
	"dueDate": "2026-03-30T01:28:23.967Z",
	"assignedToUserIds": [
	{}
	],
	"createdBy": {},
	"status": "todo",
	"priority": "low",
	"tags": [
	"string"
	],
	"roleRequired": "string",
	"completedAt": "2026-03-30T01:28:23.967Z",
	"completedBy": {},
	"createdAt": "2026-03-30T01:28:23.967Z",
	"updatedAt": "2026-03-30T01:28:23.967Z"
	}
	]

Common errors	403 access denied; 404 project not found; 500 server error

GET /api/tasks/:taskId

Returns one task by ID.

Auth	Bearer access token required and must belong to task project
What it does	Returns one task by ID.

Success response	200 ->
	{
	"_id": "string",
	"projectId": "string",
	"title": "string",
	"description": "string",
	"dueDate": "2026-03-30T01:29:31.315Z",
	"assignedToUserIds": [
	{}
	],
	"createdBy": {},
	"status": "todo",
	"priority": "low",
	"tags": [
	"string"
	],
	"roleRequired": "string",
	"completedAt": "2026-03-30T01:29:31.315Z",
	"completedBy": {},
	"createdAt": "2026-03-30T01:29:31.315Z",
	"updatedAt": "2026-03-30T01:29:31.315Z"
	}

Common errors	403 access denied; 404 task not found; 500 server error

PUT /api/tasks/:taskId

Updates task fields such as title, description, due date, assignment, status, priority, tags, and roleRequired.

Auth	Bearer access token required plus edit rights
What it does	Updates task fields such as title, description, due date, assignment, status, priority, tags, and roleRequired.

Request body	{
	"title": "string",
	"description": "string",
	"dueDate": "2026-03-30T01:30:03.128Z",
	"assignedToUserIds": [
	"string"
	],
	"status": "todo",
	"priority": "low",
	"tags": [
	"string"
	],
	"roleRequired": "string"
	}

Success response	200 ->
	{
	"message": "string",
	"task": {
	"_id": "string",

	"projectId": "string", "title": "string", "description": "string", "dueDate": "2026-03-30T01:30:03.128Z", "assignedToUserIds": [{}], "createdBy": {}, "status": "todo", "priority": "low", "tags": ["string"], "roleRequired": "string", "completedAt": "2026-03-30T01:30:03.128Z", "completedBy": {}, "createdAt": "2026-03-30T01:30:03.128Z", "updatedAt": "2026-03-30T01:30:03.128Z" } }
Common errors	400 bad tags / bad assignedToUserIds / non-member assignee; 403 no permission; 404 task not found; 500 server error
Notes	When status becomes done, completedAt and completedBy are set. When moved out of done, those fields are cleared.

DELETE /api/tasks/:taskId

Deletes a task.

Auth	Bearer access token required; Owner or task creator
What it does	Deletes a task.
Success response	200 -> {"message":"Task deleted successfully."}
Common errors	403 no permission; 404 task not found; 500 server error

Search

GET /api/search

Searches users, projects, and tasks with separate pagination for each result type.

Auth	Bearer access token required
What it does	Searches users, projects, and tasks with separate pagination for each result type.
Query params	q=<text>&type=all|users|projects|tasks&usersPage=1&projectsPage=1&tasksPage=1

Success response	200 ->
	{
	"query": "react",
	"results": {
	"users": [
	{

```
        "type": "user",
        "id": "67fd1111abcd5678ef901111",
        "displayName": "First Last",
        "email": "user@example.com",
        "profilePictureUrl": "",
        "href": "/users/67fd1111abcd5678ef901111"
      }
    ],
    "projects": [
      {
        "type": "project",
        "id": "67fd2222abcd5678ef902222",
        "name": "React Migration",
        "description": "Move frontend to React",
        "visibility": "private",
        "href": "/projects/67fd2222abcd5678ef902222"
      }
    ],
    "tasks": [
      {
        "type": "task",
        "id": "67fd3333abcd5678ef903333",
        "title": "Update navbar",
        "description": "Convert navbar to React component",
        "status": "in_progress",
        "projectId": "67fd2222abcd5678ef902222",
        "projectName": "React Migration",
        "href":
"/projects/67fd2222abcd5678ef902222?taskId=67fd3333abcd5678ef903333"
      }
    ]
  },
  "pagination": {
    "users": {
      "page": 1,
      "pageSize": 10,
      "total": 1,
      "totalPages": 1,
      "hasNextPage": false,
      "hasPrevPage": false
    },
    "projects": {
      "page": 1,
      "pageSize": 10,
      "total": 1,
      "totalPages": 1,
      "hasNextPage": false,
      "hasPrevPage": false
    },
    "tasks": {
      "page": 1,
      "pageSize": 10,
      "total": 1,
      "totalPages": 1,
      "hasNextPage": false,
      "hasPrevPage": false
    }
  }
}
```

Common errors	400 missing q or invalid type; 401 not authorized; 500 search error
Notes	Access rules: all users are searchable; only public projects and projects the requester belongs to are searchable; tasks only come from projects the requester belongs to.

Shared response and auth behavior

Behavior	What the front end should expect
Protected routes	They all run through auth middleware that expects Authorization: Bearer <accessToken>.
Unverified users	Protected routes return 403 with "Please verify your email before accessing this resource."
Revoked tokens	If tokenVersion no longer matches, protected routes return 401.
Rate limits	Auth routes include rate limiting for register, login, resend verification, verify email, and refresh.
Pagination size	Search uses a fixed page size of 10 per result type.